30. Trade-off Discussions

Key Terms:

Term	Full Name	What it does
ADR	Architecture Decision Record	Document capturing design decision rationale
SLA	Service Level Agreement	Contractual uptime/performance guarantees
FTE	Full-Time Equivalent	One person working full-time (cost measure)
MVP	Minimum Viable Product	Smallest version to test market fit
A/B Testing	Split testing	Compare two options with real users
P95/P99	Percentile latency	Latency that 95%/99% of requests are faster than

Problem Statement

Walk through a tough trade-off you would make in a system design, explaining your reasoning and how you'd validate the decision.

API Design

Trade-off Analysis APIs

Method	Endpoint	Description
`POST`	`/api/v1/analysis`	Submit a system for trade-off analysis
`GET`	`/api/v1/analysis/{id}/report`	Get analysis report
`GET`	`/api/v1/patterns?category={cat}`	List common trade-off patterns

Example: Consistency vs Availability Trade-off in APIs

```
// Option A: Strong consistency (higher latency)
GET /api/v1/inventory/{id}
```

```
  Headers:
    X-Consistency: strong
    X-Expected-Latency: 200ms

  // Option B: Eventual consistency (lower latency)
  GET /api/v1/inventory/{id}
  Headers:
    X-Consistency: eventual
    X-Expected-Latency: 20ms
    X-Max-Staleness: 5s
```

Key API Trade-offs to Discuss

Decision	Option A	Option B
Protocol	REST (simple, cacheable)	gRPC (fast, typed)
Pagination	Offset-based (simple)	Cursor-based (consistent)
Versioning	URL path (`/v1/`)	Header-based (`Accept:`)
Auth	API keys (simple)	OAuth2/JWT (secure)

Framework for Trade-off Discussions

```
STRUCTURED APPROACH:
┌─────────────────────────────────────────────────────────────────┐
│  1. IDENTIFY: What are the competing concerns?                  │
│  2. QUANTIFY: What are the costs/benefits of each option?       │
│  3. DECIDE: Which trade-off aligns with business requirements?  │
│  4. VALIDATE: How would you verify the decision was correct?    │
│  5. REVISIT: When would you reconsider this decision?           │
└─────────────────────────────────────────────────────────────────┘
```

Common Trade-off Scenarios

```
1. CONSISTENCY vs. AVAILABILITY
┌─────────────────────────────────────────────────────────────────┐
│  Scenario: E-commerce inventory during flash sale               │
│                                                                  │
│  Option A: Strong consistency                                    │
│    - Check inventory before each order                          │
│    - Reject if unavailable                                      │
│    - Risk: System slows down, orders fail                       │
│                                                                  │
│  Option B: Eventual consistency                                  │
│    - Accept order optimistically                                │
```

```
│    - Reconcile later                                          │
│    - Risk: Overselling, need to refund                        │
│                                                               │
│  My recommendation: Hybrid                                    │
│    - Reserve inventory (soft lock)                            │
│    - Confirm within timeout                                   │
│    - Release if payment fails                                 │
│    Why: Balances accuracy with availability                   │
└───────────────────────────────────────────────────────────────┘
```

2. LATENCY vs. ACCURACY

```
┌───────────────────────────────────────────────────────────────┐
│  Scenario: Search ranking with ML model                       │
│                                                               │
│  Option A: Complex model (better accuracy)                    │
│    - BERT-based ranking                                       │
│    - 200ms latency                                            │
│    - 5% higher click-through rate                             │
│                                                               │
│  Option B: Simple model (lower latency)                       │
│    - Gradient boosted trees                                   │
│    - 20ms latency                                             │
│    - Baseline click-through rate                              │
│                                                               │
│  My recommendation: Two-stage ranking                         │
│    - Fast model for candidate generation (1000 → 100)         │
│    - Complex model for final ranking (100 → 10)               │
│    Why: Best of both - low latency with good accuracy         │
└───────────────────────────────────────────────────────────────┘
```

3. COST vs. PERFORMANCE

```
┌───────────────────────────────────────────────────────────────┐
│  Scenario: Choosing between on-demand and reserved instances  │
│                                                               │
│  Option A: All on-demand                                      │
│    - $10,000/month                                            │
│    - Full flexibility                                         │
│    - No commitment                                            │
│                                                               │
│  Option B: 1-year reserved instances                          │
│    - $6,500/month (35% savings)                               │
│    - Committed capacity                                       │
│    - Less flexibility                                         │
│                                                               │
│  My recommendation: 70% reserved, 30% on-demand               │
│    - Reserve baseline load                                    │
│    - On-demand for spikes and experimentation                 │
│    Why: Predictable savings with flexibility for growth       │
└───────────────────────────────────────────────────────────────┘
```

4. BUILD vs. BUY

```
┌───────────────────────────────────────────────────────────────┐
│  Scenario: Authentication system                              │
│                                                               │
│  Option A: Build in-house                                     │
│    - Full control                                             │
```

```
│    - Customizable                                          │
│    - 3 months dev time                                     │
│    - Ongoing maintenance                                   │
│                                                            │
│  Option B: Use Auth0/Okta                                  │
│    - Quick integration (1 week)                            │
│    - Enterprise features included                          │
│    - $0.05/user/month                                      │
│    - Vendor dependency                                     │
│                                                            │
│  My recommendation: Buy (Auth0)                            │
│    - Security is hard to get right                         │
│    - Cost at 100K users = $5K/month vs. 3 FTEs to maintain │
│    - Focus engineering on core product                     │
│    Why: Auth isn't our differentiator, shouldn't build     │
└────────────────────────────────────────────────────────────┘

5. SIMPLICITY vs. SCALABILITY

┌────────────────────────────────────────────────────────────┐
│  Scenario: Starting a new project                          │
│                                                            │
│  Option A: Microservices from day 1                        │
│    - Scalable architecture                                 │
│    - Team autonomy                                         │
│    - 6 months to MVP                                       │
│    - Complex operations                                    │
│                                                            │
│  Option B: Monolith                                        │
│    - Simple development                                    │
│    - Fast iteration                                        │
│    - 2 months to MVP                                       │
│    - May need to refactor later                            │
│                                                            │
│  My recommendation: Monolith first                         │
│    - Validate product-market fit quickly                   │
│    - Easier to refactor when domain is understood          │
│    - Extract services when hitting scale limits            │
│    Why: Premature optimization is the root of all evil     │
└────────────────────────────────────────────────────────────┘
```

How to Present Trade-offs in Interviews

```
TEMPLATE:

┌────────────────────────────────────────────────────────────┐
│  "We have two options here: [A] and [B].                   │
│                                                            │
│   Option A gives us [benefit], but costs us [drawback].    │
│   Option B gives us [benefit], but costs us [drawback].    │
│                                                            │
│   Given our requirement for [specific need], I'd recommend │
│   Option [X] because [reasoning].                          │
│                                                            │
│   We can validate this by [measurement] and revisit if     │
```

```
│   [condition changes]."                                              │
└──────────────────────────────────────────────────────────────────────┘


EXAMPLE:
"We have two options for session storage: Redis and database.

Redis gives us sub-millisecond reads but costs more and adds
operational complexity.

Database is simpler and cheaper but adds 5-10ms latency per
request.

Given our requirement for <100ms page loads, I'd recommend
Redis because sessions are read on every request, and 10ms
adds up.

We can validate by A/B testing both approaches and monitoring
P95 latency. We'd revisit if Redis costs exceed 10% of
infrastructure budget."
```

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Redis	Session storage and caching trade-offs	Tech Guide
PostgreSQL	ACID transactions vs. eventual consistency	Tech Guide

---

How to Explain It Well

> **Opening**: "Every system design involves trade-offs. I use a structured approach: identify options, quantify costs/benefits, decide based on requirements, and plan to validate."

> **Key talking points**:
>
> - "No perfect solution - every choice has costs"
> - "Quantify trade-offs: 'X improves latency by 50% but costs 2x'"
> - "Align decisions with business goals, not technical preferences"
> - "Always have a validation plan and conditions to revisit"
> - "Document decisions (ADRs) for future reference"

Summary: Interview Success Tips

```
BEFORE THE INTERVIEW:
┌─────────────────────────────────────────────────────────────────┐
│  1. Practice drawing systems on whiteboard/paper                │
│  2. Know numbers: latency, throughput, storage estimates        │
│  3. Understand trade-offs for common patterns                   │
│  4. Prepare 2-3 deep dives you know well                        │
└─────────────────────────────────────────────────────────────────┘

DURING THE INTERVIEW:
┌─────────────────────────────────────────────────────────────────┐
│  1. CLARIFY: Ask questions, understand requirements             │
│  2. ESTIMATE: Back-of-envelope calculations                     │
│  3. DESIGN: High-level first, then dive deep                    │
│  4. TRADE-OFFS: Discuss alternatives and justify choices        │
│  5. EVOLVE: How would the system scale 10x?                     │
└─────────────────────────────────────────────────────────────────┘

COMMUNICATION:
┌─────────────────────────────────────────────────────────────────┐
│  ✓ Think out loud                                               │
│  ✓ Structure your answer (don't jump around)                    │
│  ✓ Use specific technologies (Redis, Kafka, not "cache")        │
│  ✓ Acknowledge trade-offs proactively                           │
│  ✓ Be honest about what you don't know                          │
└─────────────────────────────────────────────────────────────────┘
```